BASIL analysis of RNA-seq data

Federica Stolf and Lorenzo Mauri

2025-10-16

Installation

```
# Install from GitHub
devtools::install_github("federicastolf/BASIL")
```

Setup

```
library(BASIL)
# library(devtools)
# install_github("wgmao/PLIER")
library(PLIER)            # For data and comparison
library(tidyverse)
library(reshape2)
library(gridExtra)
library(ComplexHeatmap)
library(scico)
```

Data Loading and Preprocessing

We use the whole blood gene expression dataset from the PLIER package.

```
# Load whole blood gene expression data
data("dataWholeBlood", package = "PLIER")

# Load pathway information
data("bloodCellMarkersIRISDMAP", package = "PLIER")
data("canonicalPathways", package = "PLIER")
allPaths <- PLIER::combinePaths(bloodCellMarkersIRISDMAP, canonicalPathways)
cm <- intersect(rownames(dataWholeBlood), rownames(allPaths))
allPaths <- allPaths[cm,]
dataWholeBlood <- dataWholeBlood[cm,]
dim(dataWholeBlood)
#> [1] 5892   36
dim(allPaths)
#> [1] 5892  606

# Transpose and standardize (BASIL expects samples × genes)
Y <- t(dataWholeBlood)
Y <- scale(Y)

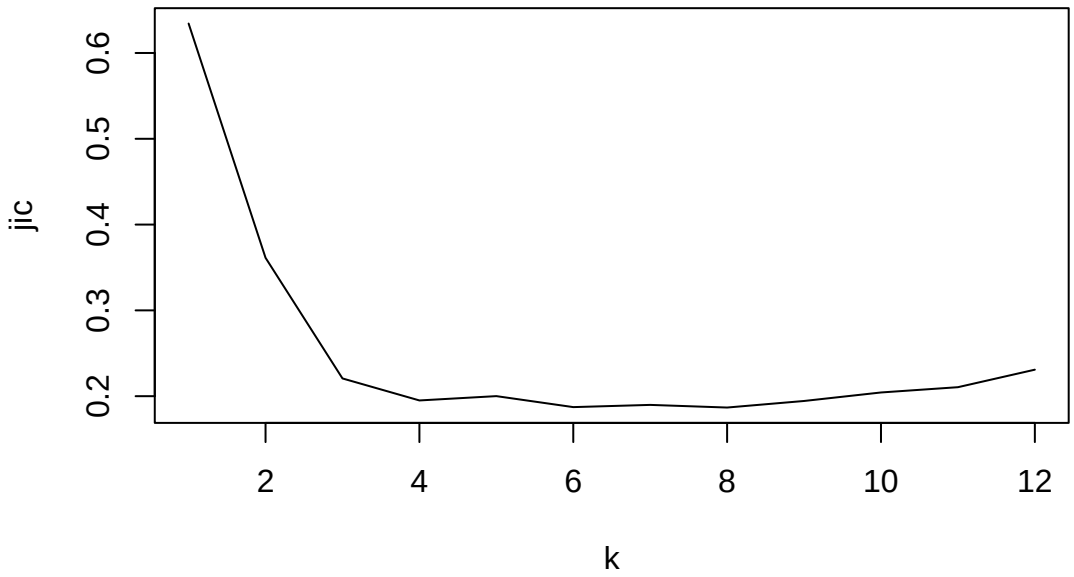
n <- nrow(Y)
p <- ncol(Y)
```

Model Fitting

Estimate Number of Latent Factors using JIC

```
svd_Y <- svd(Y)
k_max <- min(which((cumsum(svd_Y$d^2) / sum(svd_Y$d^2) > 0.8)))
k_max
#> [1] 12

# Estimate optimal k using JIC
est_k <- estimate_latent_dimension(Y, k_max = k_max)
```




```
#> [1] "k_hat =  8"
k_hat <- est_k$k_hat
```

Fit BASIL Model

```
fit_basil <- BASIL_point_estimates(Y = Y, C = allPaths, k = k_hat, v0 = 1, sigma_sq0 = 1)
#> [1] "tau_gamma  =  0.639692784285783"
#> [1] "tau_psi  =  0.222055755290138"

# Check pathway vs unstructured variance ratio
fit_basil$tau_gamma / fit_basil$tau_psi
#> [1] 2.880776

# Compute covariance: Cov = Lambda * Lambda^T + sigma_sq * I
Lambda_hat <- fit_basil$Lambda_C + fit_basil$Lambda_N
Cov_basil <- tcrossprod(Lambda_hat) + fit_basil$sigma_sq * diag(p)

Corr_basil <- cov2cor(Cov_basil)
```

Fit PLIER for Comparison

Fit PLIER with the same number of factors.

```
# PLIER expects Y as genes × samples
plier_result <- PLIER(t(Y), allPaths, k = k_hat, doCrossval = FALSE, scale = FALSE)
```

```
#> [1] 71.94351
#> [1] "L2 is set to 71.9435065426257"
#> [1] "L1 is set to 35.9717532713128"

# Compute PLIER covariance
Cov_plier <- plier_result$Z %*% cov(t(plier_result$B)) %*% t(plier_result$Z)
Corr_plier <- cov2cor(Cov_plier)
```

Uncertainty Quantification for BASIL

```
# Generate 500 posterior samples
basil_loadings_samples <- compute_posterior_samples_cc(Y = Y,
  Lambda_C = fit_basil$Lambda_C, Lambda_N = fit_basil$Lambda_N,
  tau_gamma = fit_basil$tau_gamma, tau_psi = fit_basil$tau_psi,
  sigma_sq = fit_basil$sigma_sq, P_C = fit_basil$P_C, v0 = 1,
  sigma_sq_0 = 1, n_MC = 500)
#> [1] "rho =  1.50628675310912"
```

Compute posterior samples of Γ from the posterior samples of Λ.

```
# posterior samples of Gamma
Gamma_postsamples <- compute_Gamma_samples(basil_loadings_samples$Lambda_samples,
                                           allPaths)
# Compute 95% credible intervals
ci_lowerG <- apply(Gamma_postsamples, c(1, 2), quantile, probs = 0.025)
ci_upperG <- apply(Gamma_postsamples, c(1, 2), quantile, probs = 0.975)
contains_zeroG <- (ci_lowerG < 0) & (ci_upperG > 0)

# proportions of elements of Gamma set to zero
table(contains_zeroG)/length(contains_zeroG)
#> contains_zeroG
#>     FALSE      TRUE
#> 0.2679455 0.7320545

Gamma_UQ <- apply(Gamma_postsamples, c(1, 2), mean)
Gamma_UQ[contains_zeroG] <- 0
```

Compute posterior samples of Ψ from the posterior samples of Λ.

```
# posterior samples of Psi
Psi_postsamples <- compute_Psi_samples(basil_loadings_samples$Lambda_samples,
                                       fit_basil$P_C)
# Compute 95% credible intervals
ci_lowerP <- apply(Psi_postsamples, c(1, 2), quantile, probs = 0.025)
ci_upperP <- apply(Psi_postsamples, c(1, 2), quantile, probs = 0.975)
contains_zeroP <- (ci_lowerP < 0) & (ci_upperP > 0)

# proportions of elements of Psi set to zero
table(contains_zeroP)/length(contains_zeroP)
#> contains_zeroP
#>     FALSE      TRUE
#> 0.2156314 0.7843686

Psi_UQ <- apply(Psi_postsamples, c(1, 2), mean)
Psi_UQ[contains_zeroP] <- 0
```

For the correlation analysis only focus on the 100 genes with largest variance for computational efficiency and visualization purpose.

```
gene_variances <- apply(t(dataWholeBlood), 2, var)
idx <- order(gene_variances, decreasing = TRUE)[1:100]

# Compute correlation posterior
cor_posterior <- compute_correlation_posterior_samples_cc(
  Lambda_samples = basil_loadings_samples$Lambda_samples[idx,,],
  sigma_sq_samples = basil_loadings_samples$sigma_sq_samples,
  samples = TRUE
)

CorrBasil_mean <- cor_posterior$posterior_mean

CorrBasil_mean_zeroed <- CorrBasil_mean
# Compute 95% credible intervals
corrBasil_qs <- apply(cor_posterior$posterior_samples, c(1, 2), quantile,
  probs = c(0.025, 0.975))
CorrBasil_mean_zeroed[((corrBasil_qs[1,,] <0) & (corrBasil_qs[2,,] >0))] = 0
# Proportion of correlations set to zero
mean(CorrBasil_mean_zeroed==0)
#> [1] 0.8664
```

Visualization

Correlation Heatmaps

Compare correlation estimates across methods.

```
Y_subset <- Y[,idx]
# empirical correlation
Corr_empirical <- cor(Y_subset)

cols <- scico(51, palette = "vik")
breaks <- seq(-1, 1, length.out = 52)

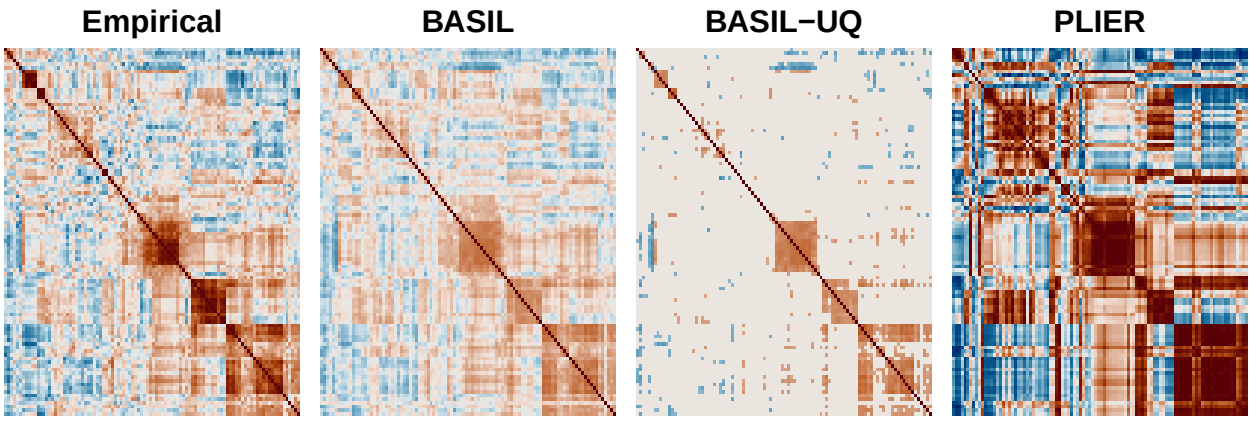
# empirical correlation
emp_corr_plot <- plot_correlation_heatmap(Corr_empirical, title = "Empirical",
  cluster = TRUE, breaks = breaks)
row_order <- emp_corr_plot$tree_row$order
col_order <- emp_corr_plot$tree_col$order

# BASIL
row.names(CorrBasil_mean) <- colnames(Y_subset)
colnames(CorrBasil_mean) <- colnames(Y_subset)
basil_corr_plot <- plot_correlation_heatmap(CorrBasil_mean, title = "BASIL",
  row_order = row_order, col_order = col_order, breaks = breaks)

# BASIL UQ
row.names(CorrBasil_mean_zeroed) <- colnames(Y_subset)
colnames(CorrBasil_mean_zeroed) <- colnames(Y_subset)
basilUQ_corr_plot <- plot_correlation_heatmap(CorrBasil_mean_zeroed,
  title = "BASIL-UQ", row_order = row_order, col_order = col_order,
  breaks = breaks)
```

```
# PLIER
sub_corrPLIER <- Corr_plier[idx,idx]
plier_corr_plot <- plot_correlation_heatmap(sub_corrPLIER, title = "PLIER",
  row_order = row_order, col_order = col_order, breaks = breaks)
```

```
corrWB <- grid.arrange(emp_corr_plot$gtable, basil_corr_plot$gtable,
  basilUQ_corr_plot$gtable, plier_corr_plot$gtable, ncol = 4)
```

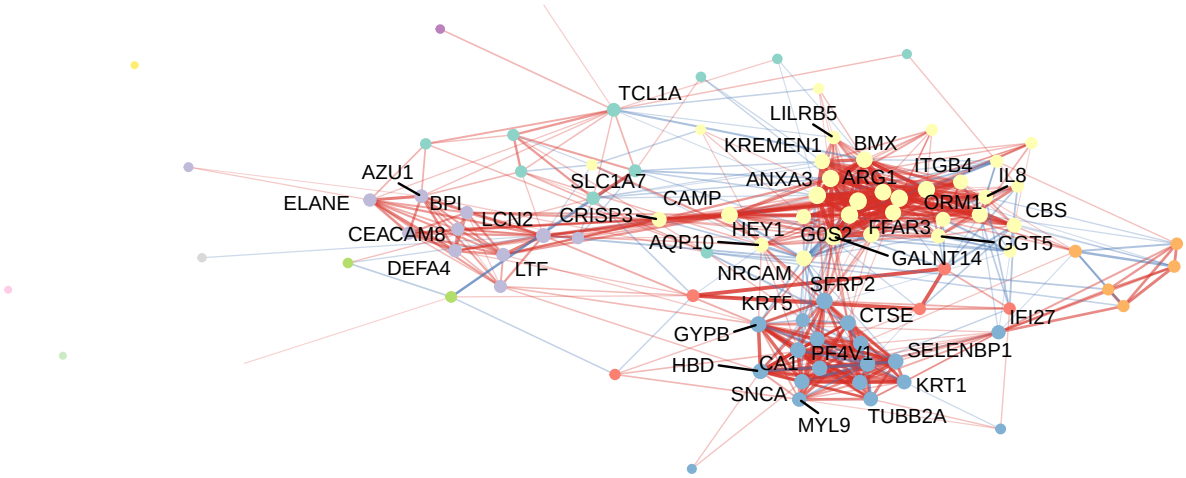


Gene network plot

```
df_long <- melt(CorrBasil_mean[]) %>%
  filter(value < 0.9999)
df_sorted <- df_long %>%
  arrange(desc(value))

network_plot <- plot_gene_network(CorrBasil_mean_zeroed, n = 50)
network_plot
```

Gene co−expression network



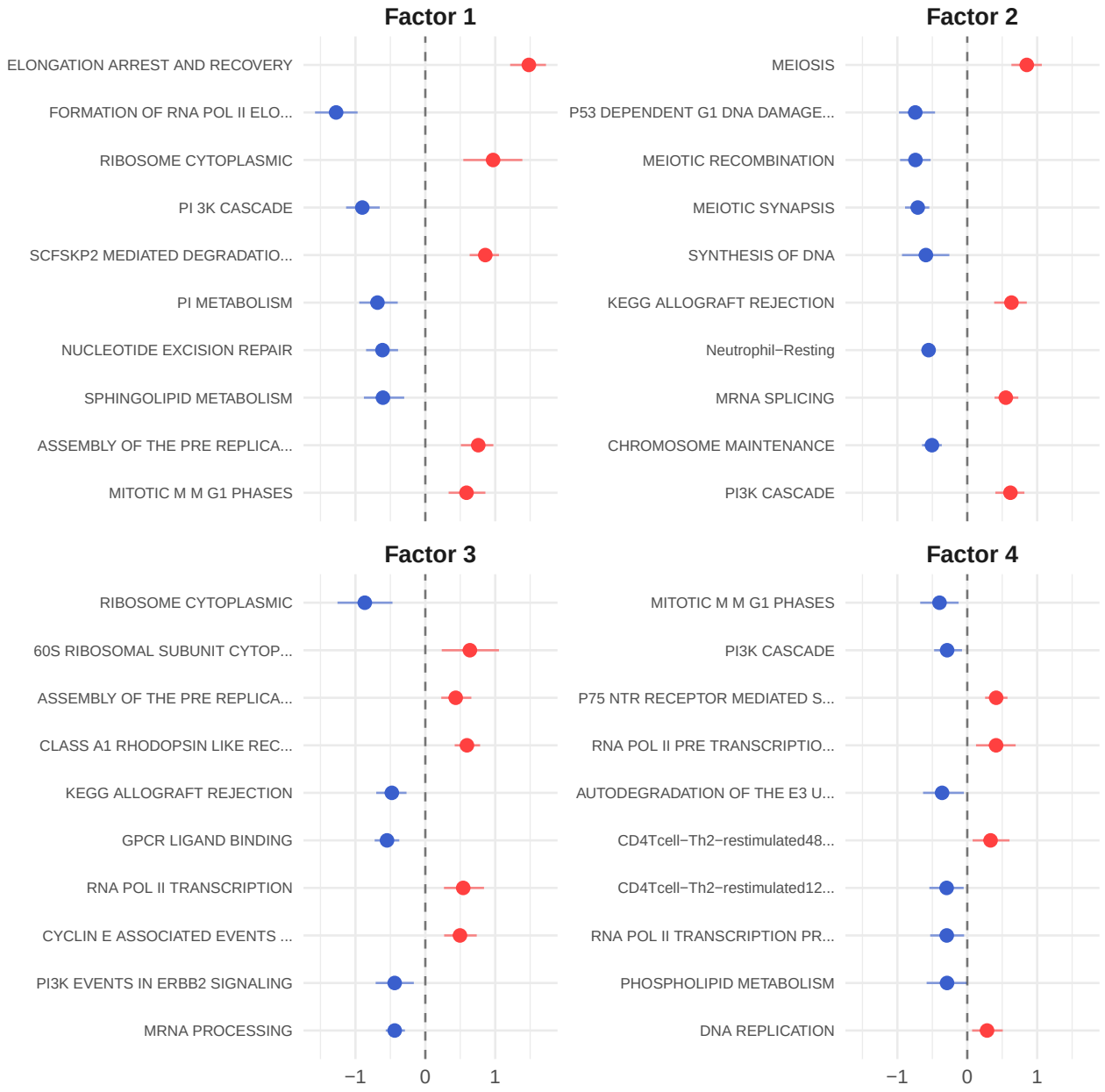
Factor annotation

This visualization highlights the pathways most strongly associated with each latent factor, with credible intervals indicating the statistical significance of each link.

```
p_names <- colnames(allPaths)
# consider the first 4 factors
idf_fa <- c(1:4)

# get top pathways of Gamma for each factor
data_pplot <- get_topPathways(Gamma_UQ[,idf_fa], p_names, top_n = 10,
                              clean_names = TRUE, ci_lower = ci_lowerG[,idf_fa],
                              ci_upper = ci_upperG[,idf_fa], max_length = 30)

fplot <- dotplot_top_pathways(data_pplot)
fplot
```



This visualization highlights the genes most strongly linked to the unstructured variability component, representing variance not captured by known pathways.

```
normPsi <- apply(Psi_UQ, 1, function(x) sqrt(sum(x^2)))

# focus on the top 10 genes
top_Psi <- order(normPsi, decreasing = TRUE)[1:10]
genePsi <- rownames(allPaths[top_Psi,])
normPsi_top <- normPsi[top_Psi]
genePsi_data <- data.frame(gene = genePsi, row_norm = normPsi_top)
genePsi_data <- genePsi_data %>%
  arrange(desc(row_norm)) %>%
  mutate(gene = factor(gene, levels = rev(gene)))

# Create the plot
```

```r
psi_plot <- ggplot(genePsi_data, aes(x = row_norm, y = gene)) +
  geom_bar(stat = "identity", fill="steelblue3") +
  labs(x = "Row Norm", y="") +
  coord_cartesian(xlim = c(0.9, NA)) +
  theme_minimal(base_size = 11) +
  theme(axis.text.y = element_text(size = 12),
    axis.text.x = element_text(size = 10),
    axis.title = element_text(size = 11),
    legend.position = "none",
    panel.grid.major.y = element_blank(),
    panel.grid.minor = element_blank())
psi_plot
```

